Formula 1 Season Simulator — Product Requirements Document

Status: Draft v0.2 **Prepared:** 7 October 2026 **Project team:** Two students **Implementation constraint:** Java 6

1. Product summary

Build a desktop Formula 1 season simulator in Java 6. A user can inspect the 2026 teams and drivers, see team strength and sponsors, run regular qualifying and Grand Prix race sessions across all rounds, and review results and championship standings. The project has less than three weeks and must favor a small, finishable implementation.

The simulator is an educational, data-driven model. It should produce believable, explainable variation in lap times and outcomes. It cannot promise exact real-world results or telemetry-level accurate lap times without licensed historical telemetry and a substantially larger physics/data effort.

2. Goals

- Represent all 11 teams and 22 pre-season race drivers confirmed for the 2026 grid.
- Store and present team, driver and sponsor data, including team and driver images/logos.
- Model team performance using editable, documented statistics.
- Model sponsor funding and its bounded team-performance modifier.
- Simulate all scheduled 2026 Grand Prix rounds using regular qualifying and Sunday race sessions, points and standings.
- Keep the implementation achievable for two students using Java 6 and SQLite, with a delivery deadline under three weeks.

3. Out of scope for the first release

- Full vehicle dynamics, tire thermodynamics, aerodynamic simulation or real-time racing graphics.
- Exact reconstruction of real 2026 qualifying and race results.
- Live timing, online multiplayer, network services, user accounts or cloud hosting.
- Sprint sessions, practice sessions, reserve-driver substitutions, mid-season driver transfers and historical seasons.
- Reproducing commercial F1 broadcast graphics or using unlicensed assets.

These can be considered extensions after the core season works.

4. Users and primary use

Primary user: A student or F1 fan running a season and comparing teams/drivers.

Main flow: Open the application → inspect teams/drivers → start a new 2026 season → run qualifying for the next event → inspect grid → run race → inspect results and updated standings → continue until the championship is complete.

5. Scope and data assumptions

5.1 Initial 2026 roster

The seed dataset will contain the 11 teams and pre-season lineups below. The exact spelling, branding and roster should be checked against the official F1/FIA references at implementation time. For this project, the season roster is a fixed snapshot; changes during the real season are not automatically imported.

Team	Drivers
McLaren	Lando Norris, Oscar Piastri
Mercedes	George Russell, Kimi Antonelli
Red Bull Racing	Max Verstappen, Isack Hadjar
Ferrari	Charles Leclerc, Lewis Hamilton
Williams	Carlos Sainz, Alexander Albon
Racing Bulls	Liam Lawson, Arvid Lindblad
Aston Martin	Fernando Alonso, Lance Stroll
Haas	Esteban Ocon, Oliver Bearman
Audi	Nico Hülkenberg, Gabriel Bortoleto
Alpine	Pierre Gasly, Franco Colapinto
Cadillac	Sergio Pérez, Valtteri Bottas

The official F1 2026 roster and team pages are the source of truth for names and team membership: F1 drivers, F1 teams. The officially announced season calendar should be used for event names, order and circuits: F1 2026 calendar announcement.

5.2 Asset policy

Store local image files (or relative asset paths) and metadata in the database; avoid embedding large image binaries in database rows. Each driver may have a portrait and each team a logo. The application must show a neutral placeholder when an image is missing. Use assets only when the project has permission to use them; record source and attribution/license in an asset table or accompanying credits file. Brand/team logos and driver portraits are not presumed free to redistribute.

5.3 Sponsor policy

Each team has five sponsors by default, with support for up to seven. Sponsor identity, cash contribution and the calculated modifier are stored separately. Financial figures are simulation inputs, not claims about confidential real sponsorship contracts. A sponsor can be based on a real brand name, but use invented contribution amounts unless reliable, licensed/public figures are available.

Proposed modifier rule (to confirm in implementation): Each sponsor's contribution adds to a team-level sponsor modifier using a documented tier or linear formula; the *combined sponsor modifier is capped at +10%*. Larger contributions increase the modifier, up to the cap. The modifier affects team performance only; sponsor money is not directly awarded to drivers. Do not stack individual sponsor bonuses. Store the base contribution and calculated contribution effect so the rule remains transparent.

Suggested configurable starting model: `teamSponsorModifier = min(0.10, totalEligibleSponsorContribution / contributionForMaxBonus * 0.10)`. Use decimal fractions internally (0.10 = 10%). Set the threshold from the chosen fictional seed data and document it. This is a game-balance rule, not an assertion about actual F1 sponsor economics.

6. Functional requirements

FR-1 — Team and driver directory

- List all 11 teams and their two initial race drivers.
- Display team name, logo, team performance attributes, driver names and portraits.
- Show a fallback image if a path is unavailable.
- Support lookup by team or driver name.

FR-2 — Persistent data

- Save teams, drivers, sponsors, circuits, season events, sessions, results and standings in a relational database.
- Use stable IDs and foreign keys; avoid relying on display names as identifiers.
- On first launch, initialize the supplied seed data without duplicating it on later launches.
- Keep database access behind a small data-access layer so UI and simulation rules do not issue raw SQL throughout the application.

FR-3 — Team and driver performance

- Give each team an editable pace rating on a documented scale (for example 0–100). Other team stats are optional.
- Give each driver editable qualifying pace, race pace, consistency and experience ratings (for example 0–100).
- Higher experience and consistency reduce the chance of a mistake; qualifying/race pace ratings affect lap times. Ratings are balancing inputs, not official measurements.
- Keep rating effects simple, documented and bounded. Team pace may be included as an additional lap-time modifier if time allows.

FR-4 — Sponsors and funding

- Support five to seven sponsors per team.
- Record sponsor name, contribution amount, currency, team association and modifier contribution.
- Calculate and display the team-level modifier, with a hard maximum of +10%.
- Keep calculation parameters centralized/configurable and show the resulting bonus in the team view.

FR-5 — Season calendar

- Load all scheduled 2026 Grand Prix rounds in correct order with round, event, circuit, location and race lap count.
- Allow the season to advance one event at a time; prevent duplicate session results.
- The simulator includes Saturday qualifying and Sunday Grand Prix only. Sprint and practice sessions are excluded.
- Use the official F1 calendar as the source for round order and verify any calendar changes before delivery.

FR-6 — Qualifying

- Simulate one regular Saturday qualifying session per Grand Prix and produce a classified result for all 22 drivers.
- Model Q1/Q2/Q3 elimination (22 → 16 → 10); a minimal implementation may use one generated timed lap per phase rather than simulating every real attempt.
- Each driver's best valid lap determines their grid position. Driver experience/consistency influences mistake probability.
- Display times in `m:ss.mmm` and gaps to pole. Support a fixed random seed for reproducible runs.

FR-7 — Race and lap times

- Simulate each Grand Prix distance and produce a finish order, status, total time, fastest lap and points.
- Establish a circuit benchmark qualifying lap and race-pace lap in seconds. Example only: Monaco qualifying 75.000 seconds and race pace 80.000 seconds. Generate driver lap times as a simple, documented adjustment to the applicable benchmark using driver stats (and team pace if implemented), plus small bounded variation.
- Keep the initial core model simple: weather, fuel burn, traffic and pit strategy do not need to affect lap times. Do not require tire degradation for the first working version.
- Experience and consistency affect a per-lap mistake chance. A mistake may add a configurable time penalty (e.g. lockup) or cause a retirement (e.g. crash), with low bounded probability.
- Persist per-lap times when practical; otherwise persist the classification, best lap and race total. Do not let lap-by-lap persistence delay completion of the core season.
- Apply the configured 2026 points rule and store the scoring-rule version with the season.

FR-8 — Standings and season completion

- Update driver and constructor/team points after each race.
- Display event results, current driver standings and team standings.
- Resolve ties using a documented countback rule (wins, then seconds, etc.).
- At season completion, show champion and final standings.

FR-9 — Basic usability and errors

- Provide clear actions to run qualifying and race sessions.
- Confirm before starting a new season over an existing one.
- Report database or missing asset errors clearly; do not silently corrupt or duplicate results.

7. Non-functional requirements

- **Java compatibility:** Compile and run with Java 6. Confirm SQLite JDBC driver support against the actual runtime early.
- **Platform/UI:** Desktop UI is assumed. Swing is the lowest-risk Java 6 choice given prior experience; use a clean layout, image panels, consistent colors and readable typography. A richer visual appearance should not require a newer toolkit or jeopardize the schedule.
- **Performance:** A full event simulation for 22 drivers should complete within a few seconds on an ordinary school computer.
- **Reliability:** A saved season must survive closing and reopening the program.
- **Reproducibility:** Store season settings and random seed when fixed-seed mode is used.
- **Maintainability:** Separate domain model, persistence, simulation engine and UI. Keep constants and balance formulas documented and centralized.
- **Data integrity:** Validate required fields, unique driver/team identities, sponsor count limits and modifier cap.

8. Proposed lap-time and tire model

The lap-time owner can implement the simplest model first: `lapSeconds = circuitBenchmarkSeconds + driverAdjustmentSeconds + smallRandomVariationSeconds`. Qualifying and race pace use separate benchmark values. A higher pace rating should yield a lower adjustment. If sponsor/team pace modifiers are included, keep their formula separate and apply it consistently. Store and calculate times in seconds; format only for display.

Tire degradation is an optional extension, not a release dependency. If included, implement three compounds (Soft, Medium, Hard) with configurable initial pace offsets and a simple per-lap degradation amount. Add the wear penalty each lap: `lapSeconds = baseLapSeconds + compoundOffset + wearPerLap * tireAgeLaps`. Soft should start fastest and wear fastest, Hard slowest and wear slowest, Medium between them. A pit stop resets tire age and changes compound. Circuit-specific stint lengths or Pirelli data can later tune these values, but do not make external tire-life data or strategy optimization a prerequisite. Use only public, attributable data; real stint length varies with conditions and cannot be represented as a guaranteed life limit.

Experience and consistency should reduce mistake probability through a transparent bounded formula. Mistakes can add a time penalty, invalidate a qualifying lap, or retire a driver. Use a fixed seed during tuning. Aim for plausible distributions rather than exact predictions.

9. Data model (initial proposal)

- `Team(team_id, name, short_name, logo_path, pace_rating)`
- `Driver(driver_id, team_id, name, nationality, number, portrait_path, quali_rating, race_rating, consistency_rating, experience_rating)`
- `Sponsor(sponsor_id, name, contribution_amount, currency, source_note)`
- `TeamSponsor(team_id, sponsor_id, contribution_amount, modifier_contribution)`
- `Circuit(circuit_id, name, country, lap_count, length_km, reference_lap_seconds)`
- `Season(season_id, year, random_seed, scoring_rule_version, status, save_slot_name)`
- `Event(event_id, season_id, round_number, circuit_id, status)`
- `Session(session_id, event_id, type, status)`
- `SessionResult(session_id, driver_id, position, best_lap_seconds, total_seconds, gap_seconds, status, points, fastest_lap)`
- Optional `LapResult(session_id, driver_id, lap_number, lap_seconds, tire, pit_lap, notes)`
- `Asset(asset_id, owner_type, owner_id, path, source_url, license_or_permission, attribution)`

Use SQLite as the proposed embedded database and confirm a JDBC driver compatible with the school Java 6 runtime early. If multiple save slots are implemented, each slot should represent a separate Season record or a separate database file, with a simple create/load/delete flow. For the minimum release, save/resume one season; multiple slots are a stretch goal. Store image paths relative to the application data folder and back up the database plus asset directory together.

10. Acceptance criteria

The first release is acceptable when:

1. It loads and displays the full 11-team, 22-driver seed roster and available images/logos.

2. Each team can have 5–7 sponsors and the calculated modifier never exceeds 10%.

3. It can run regular qualifying and a complete Grand Prix for every scheduled 2026 round; there are no practice or sprint sessions.

4. Session results include ordered classifications and readable lap/total times; race points correctly update driver and team standings.

5. A season can be saved, closed, reopened and continued without losing or duplicating completed sessions.

6. Lap times use documented circuit benchmarks and driver ratings; experience/consistency affect mistake probability; fixed-seed mode reproduces the same results.

7. The project compiles and runs with Java 6 and SQLite in the agreed environment, with setup instructions and asset attribution included. Multiple save slots are a stretch goal; one persistent save/resume flow is required.

11. Risks and mitigations

Risk	Mitigation
Lap-time and tire modeling expands beyond the deadline	Ship benchmark-plus-driver-rating lap times first; make tire wear optional and keep conditions/strategy out of the core.
Java 6 is old and modern SQLite JDBC libraries may not support it	Confirm compiler/runtime and JDBC options immediately; keep dependencies minimal.
Real sponsor payments are confidential or inconsistent	Use clearly labeled estimated/fictional contributions and make all figures editable.
Driver/team imagery has usage restrictions	Track source and permission; use placeholders or self-created graphics when reuse rights are unclear.
Scope becomes too large for two students	Deliver roster, persistence, qualifying, race, standings first; defer weather/strategy complexity and sprint weekends.
Uncalibrated ratings create implausible outcomes	Keep ratings visible and tunable; compare output distributions with public historical results and adjust.

12. Open decisions

1. Confirm the exact Java 6 runtime and verify the SQLite JDBC driver on it.

2. Decide whether to add per-lap tire wear after benchmark-based laps and the complete season work.

3. Choose estimated/fictional sponsor amounts and rating values for balance.

4. Decide which image/logo sources and licenses are acceptable.

13. Delivery priority for a three-week project

1. First: agree the schema/data contract, Java 6 runtime, SQLite JDBC option and lap-time inputs.

2. Core: roster/calendar database, directory UI, qualifying, race classification, points and standings.

3. Persistence: save and resume one season.

4. Polish if time remains: better Swing visuals, images/logos, multiple save slots, sponsor display/modifier, tire degradation.

The exact cut line should be agreed by the two students. Multiple save slots and tire wear are explicitly stretch goals; the core should remain demonstrable without them.

14. Sources

- Formula 1 — 2026 drivers
- Formula 1 — 2026 teams
- Formula 1 — 2026 calendar announcement
- Optional tire-model background: Pirelli F1 calendar and circuit notes. Use stint-life information only if public, attributable circuit-specific data can be found; it is not needed by the core model.
- FIA rules and sporting details should be checked from the current official FIA Formula One regulations before implementing points and qualifying.